

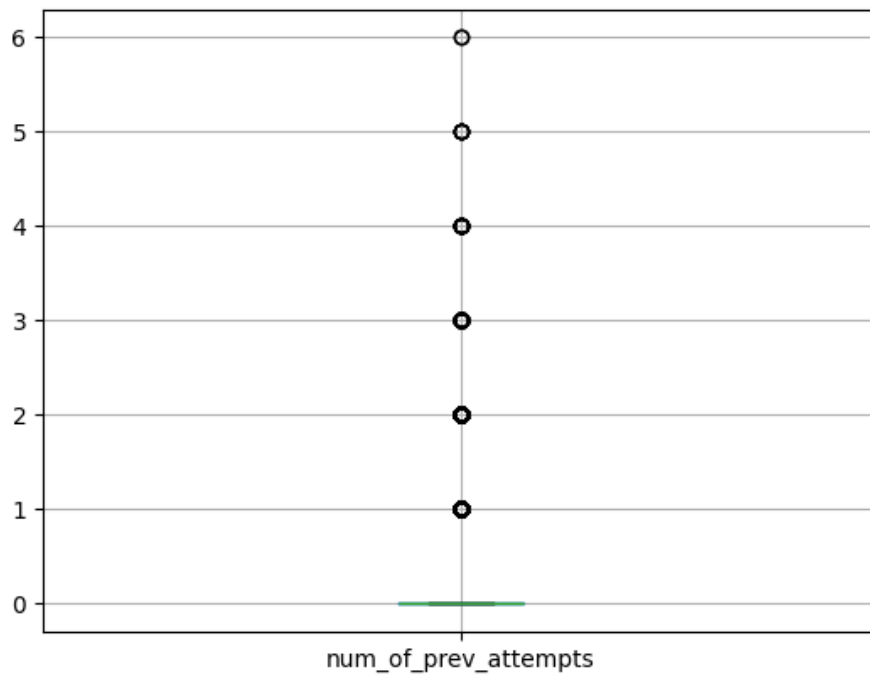
Documentatie Tema 1 IA

Antonie Aureliu Valentin

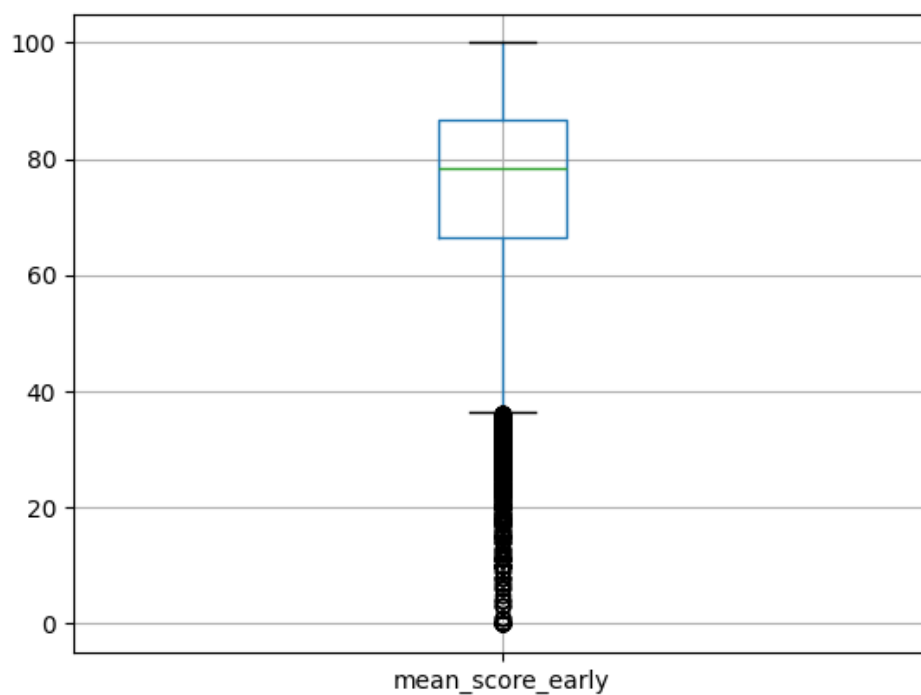
statistici attribute numerice continue

feature	non_null_numeric	average	std_dev	min	max	Percentile 25	Percentile 50
num_of_prev_attempts	18802	0.17	0.48	0.0	6.0	0.0	0.0
studied_credits	18802	80.44	37.06	30.0	430.0	60.0	60.0
date_registration	18799	-67.87	47.85	-311.0	167.0	-98.0	-54.0
date_unregistration	3542	111.39	62.19	196.0	444.0	59.0	108.0
module_presentation_length	18802	256.03	13.33	234.0	269.0	241.0	262.0
n_assessments_early	18787	2.66	1.15	1.0	6.0	2.0	2.0
mean_score_early	18770	74.36	17.16	0.0	100.0	66.5	78.33
weighted_mean_score_early	18780	73.43	16.57	0.0	100.0	64.7	76.0
final_coursework_score	18802	70.46	16.85	0.0	100.0	61.0	73.12
total_clicks_early	18802	791.46	860.59	0.0	12783.0	262.0	523.0
active_days_early	18802	35.61	22.45	0.0	115.0	18.0	31.0
clicks_dataplus	18742	0.03	0.54	0.0	31.0	0.0	0.0
clicks_dualpane	18742	0.63	2.38	0.0	58.0	0.0	0.0
clicks_externalquiz	18742	1.32	4.98	0.0	337.0	0.0	0.0
clicks_forumng	18742	194.11	360.66	0.0	6355.0	24.0	89.0
clicks_glossary	18742	1.72	14.66	0.0	564.0	0.0	0.0
clicks_homepage	18742	152.23	179.56	0.0	7075.0	56.0	106.0
clicks_htmlactivity	18742	0.34	1.43	0.0	25.0	0.0	0.0
clicks_oucollaborate	18742	1.9	4.48	0.0	110.0	0.0	0.0
clicks_oucontent	18742	197.97	291.11	0.0	4914.0	14.0	83.0
clicks_ouilluminate	18742	0.99	5.31	0.0	196.0	0.0	0.0
clicks_ouwiki	18742	10.18	22.8	0.0	372.0	0.0	0.0
clicks_page	18745	31.35	167.02	0.0	1034.87	0.0	0.0
clicks_questionnaire	18742	1.1	3.02	0.0	28.0	0.0	0.0
clicks_quiz	18742	113.52	203.61	0.0	4535.0	0.0	39.0
clicks_repeatactivity	18742	0.0	0.04	0.0	4.0	0.0	0.0
clicks_resource	13016	24.21	63.83	0.0	4532.0	7.0	16.0
clicks_sharesubpage	18742	0.01	0.1	0.0	6.0	0.0	0.0
clicks_subpage	18742	78.39	97.69	0.0	4150.0	18.0	45.0
clicks_url	13209	14.0	25.2	0.0	1732.0	3.0	8.0
submission_rate_early	18787	0.44	0.19	0.17	1.0	0.33	0.33
refs_oucontent	18742	193.93	285.34	-22.5	4816.53	14.53	81.64
macro_stats	18802	0.0	1.01	-3.85	3.94	-0.69	0.0

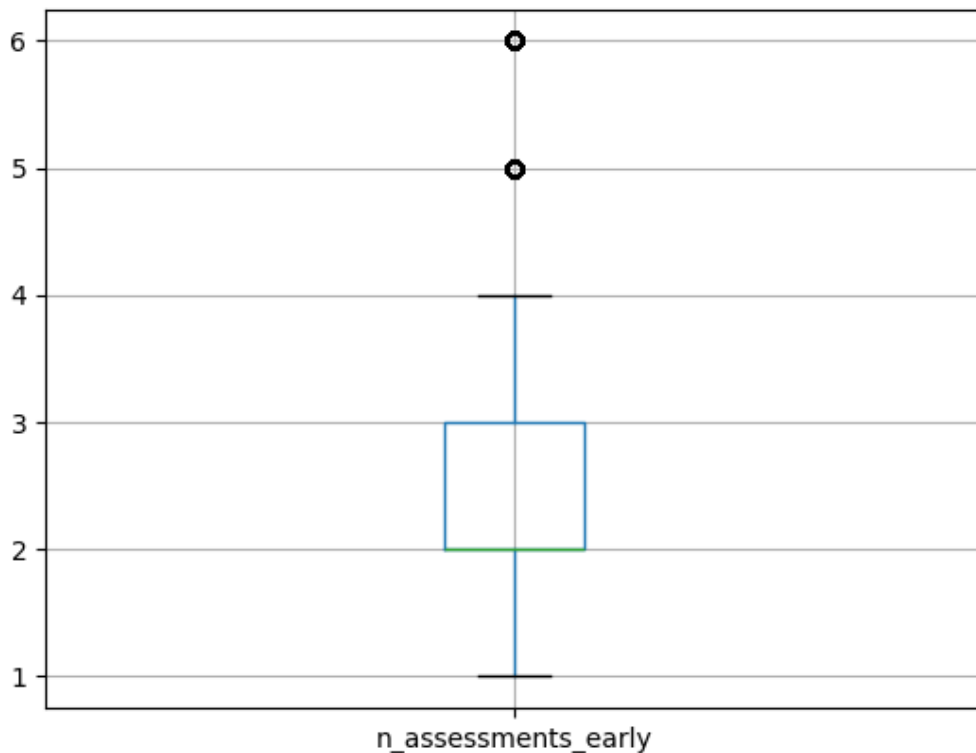
Tabelul de mai sus ofera o perspectiva importanta asupra datelor, care ne vor ajuta in prelucrarile viitoare, spre exemplu la eliminarea outlierilor. Un lucru important de vazut momentan este scalele valorice semnificativ diferite. Acest lucru va afecta semnificativ antrenarea, fiind necesara standardizarea datelor.



Se observa distributia numeric discreta a atributului num_of_prev_attempts, in multimea {0, 1, 2, 3, 4, 5, 6}, precum si predominanta valorii 0. Punctele negre – valorile {1,2,3,4,5,6} reprezinta outliers.



Se observa distributia numeric continua a atributului mean_score_early. Punctele negre reprezinta outlieri. Acest grafic este foarte important pentru observarea distributiei (dreptunghiul albastru reprezinta zona unde se afla jumatate din valori, iar linia verde reprezinta centrul distributiei).

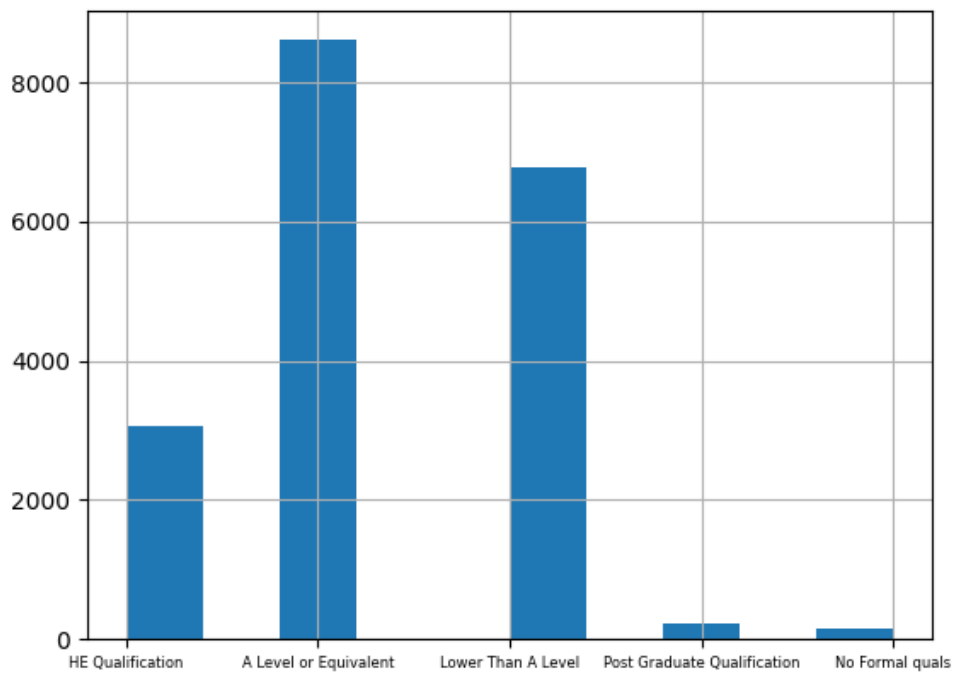


Se observa distributia numeric discreta a atributului n_assessments_early, in multimea {1, 2, 3, 4, 5, 6}. Se observa si mediana egala cu 2, precum si zona unde se afla jumate din valori. Punctele negre {5, 6} reprezinta outlieri.

statistici attribute ordinale

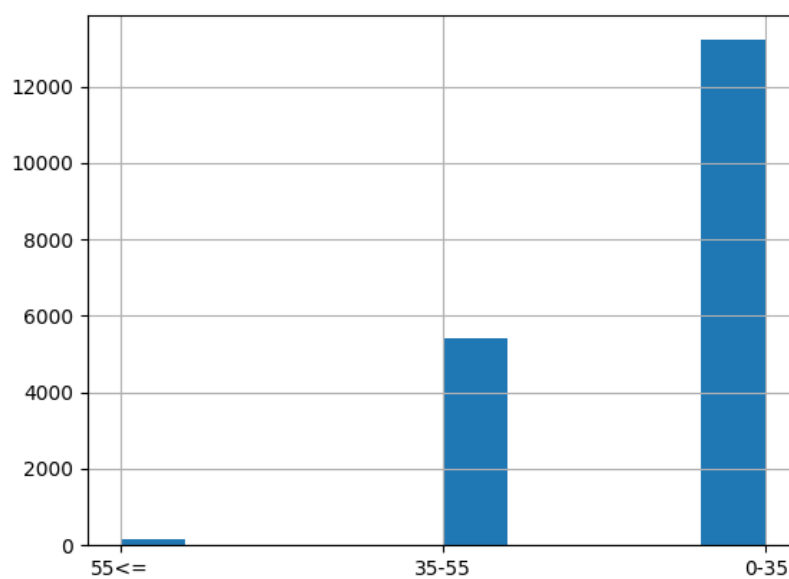
feature	non_null_discrete	unique
code_module	18802	6
code_presentation	18802	4
gender	18802	2
region	18802	13
highest_education	18802	5
imd_band	18028	10
age_band	18802	3
disability	18802	2
final_result	18802	4
clicks_freq_init	18742	3

Se observa ca numarul de etichete unice este mic pentru fiecare atribut.



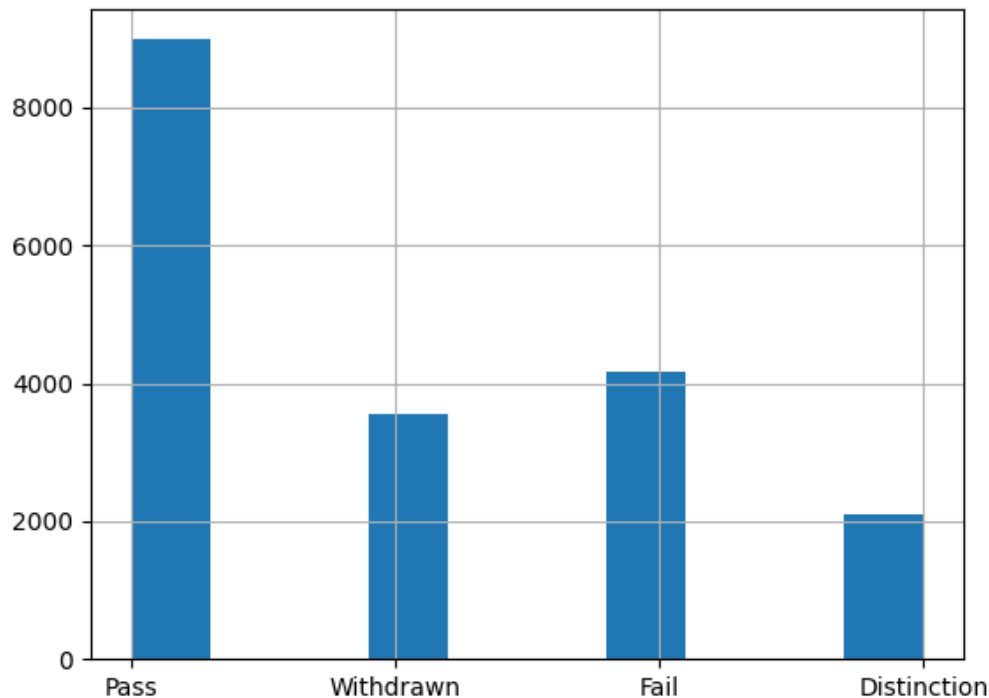
highest_education

Se observa faptul ca graficul este asimetric, iar distributia este una reala, asteptata, confirmand tendintele pietei.



age_band

Se observa, la fel, o distributie reala, asteptata, majoritatea studentilor avand varsta in intervalul [0-35].



final_result

Se observa distributia obiectivului de clasificare, final_result. Aceasta distributie m-a ajutat foarte mult in cele ce urmeaza, in analizele de corelatie.

Analiza echilibrului de clase

1. analiza intre un atribut numeric (num_of_prev_attempts) si sarcina de clasificare (final_result)

Deoarece final_result este categorial, am transformat (map) valorile in numerice (Withdrawn -> 0, Fail -> 1, Pass -> 2, Distinction -> 3). Nu am folosit LabelEncoder, deoarece ia etichetele in ordine alfabetica si nu ar fi fost o ordine valida. Spre exemplu, Distinction ar trebui sa aiba o valoare mai mare decat Fail, nu invers.

Pentru ca acum, num_of_prev_attempts si final_result sunt numerice, pot aplica metoda de corelatie Pearson.

Coeficientul Pearson, in acest caz, este **-0.105**, ceea ce semnifica o corelatie negativa foarte slaba.

2. analiza intre un atribut categorial (age_band) si sarcina de clasificare (final_result)

Pentru ca ambele sunt categoriale, am folosit testul chi-patrat.

Valoarea p reprezinta probabilitatea ca diferentele observate intre datele reale si cele asteptate sa fi aparut din intamplare, presupunand ca ipoteza nula (nu exista nicio asociere) este adevarata.

Deoarece valoarea p este de $4.36 * 10^{-25} \ll 0.05$, inseamna ca datele sunt dependente si exista o asociere semnificativa intre acestea. Acest lucru este de asteptat si reprezinta o statistica reala.

3. analiza intre un atribut numeric (num_of_prev_attempts) si sarcina de regresie (final_coursework_score)

Pentru ca ambele sunt numerice, pot sa folosesc metoda Pearson direct.

Coeficientul Pearson, in acest caz, este **-0.09**, ceea ce semnifica o corelatie negativa foarte slaba.

4. analiza intre un atribut categorial (age_band) si sarcina de regresie (final_coursework_score).

Pentru a putea aplica testul Pearson, am transformat atributul categorial intr-unul numeric, folosind LabelEncoder. In acest caz, fata de cel de mai sus, ordinea alfabetica a label-urilor nu a afectat negativ testul. Acest lucru este din noroc, deoarece ordinea alfabetica se suprapune firesc peste ordinea cronologica reala.

Coeficientul Pearson, in acest caz, este **0.06**, ceea ce semnifica o corelatie pozitiva foarte slaba.

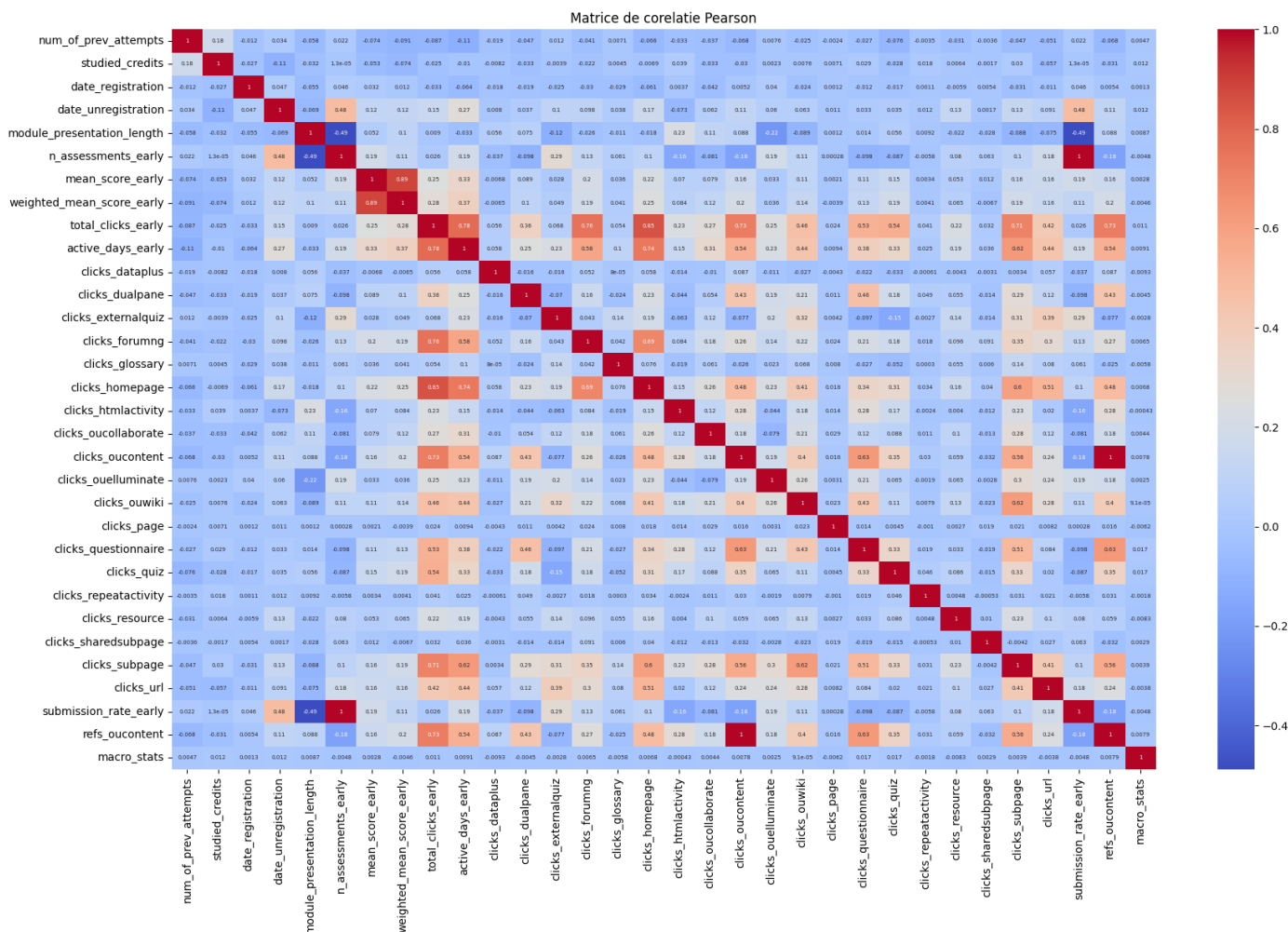
Se observa faptul ca exista o dependenta intre age_band si promovabilitate, dar nu intre age_band si punctajul final.

Matricea de corelatie Pearson

Mai jos, urmeaza afisarea matricei de corelatie Pearson, precum si o privire mai atenta asupra dependentei si corelatiei atributelor.

In graficul de mai jos, marcajul cu rosu reprezinta corelatie pozitiva, iar marcajul cu albastru reprezinta corelatie negativa. Mai putem observa ca o nuanta cu cat este mai inchisa, cu atat coeficientul este mai mare (in modul), sau mai bine spus corelatia este mai puternica.

In analiza ce urmeaza, trebuie sa ne ghidam dupa mai multi factori. In primul rand, corelatia Pearson functioneaza numai pentru attribute numerice, deci nu categoriale. Trebuie tinut cont si de corelatia cu obiectivul, in eliminarea atributelor redundante. Spre exemplu, in teorie, daca doua attribute sunt puternic corelate, eliminam atributul care este mai putin corelat cu obiectivul (final_result_encoded – encodat ca sa fie numeric). In schimb, aceasta eliminare trebuie sa se faca si in concordanta cu algoritmul folosit, anumiti algoritmi sunt mai sau mai putin sensibili la astfel de modificari.



Observam ca perechile de attribute (n_assessments_early, submission_rate_early) si (clicks_oucontent, refs_oucontent) sunt puternic corelate (coeficientul de corelatie > 0.95).

Coeficient n_assessments_early – corelatie cu obiectivul 0.24152243863236839

Coeficient submission_rate_early – corelatie cu obiectivul 0.24152243860075207

Putem observa ca este aproximativ la fel.

Coeficient clicks_oucontent – corelatie cu obiectivul 0.2010014339855086

Coeficient refs_oucontent – corelatie cu obiectivul 0.20111708681371676

Putem observa ca este aproximativ la fel.

„!Atenție: Nu ne dorim să eliminăm attribute corelate cu obiectivele noastre. Există situații de overfitting în care apare o relație de corelare între un atribut și coloana obiectiv, care nu generalizează pe setul de validare, moment în care putem elimina

acel atribut. Însă, în general ne dorim attribute sau grupări de attribute care să fie corelate cu sarcina, pentru a prezice pe baza unor trenduri din date.” – disclaimer din enunțul temei referitor la ce am spus mai sus. Acesta este motivul central pentru care am ales să nu elimin attributele redundante. Un alt motiv este că folosesc RandomForestClassifier și RandomForestRegressor, iar această eliminare nu mă ajută. Acest lucru o să-l detaliez în secțiunea în care o să discut despre algoritmi. În plus, pe baza testelor proprii, chiar îmi scade puțin precizia (clasificare) și MSE (regresie).

Observăm că atributul macro_stats nu este corelat semnificativ cu niciun alt atribut, mai mult, nu este corelat nici cu obiectivul (final_result_encoded - 0.0001 și final_coursework_score - -0.0061). De aceea, am ales să elimin această coloană. Am considerat că nu mă ajută și este „noise”.

Preprocesarea datelor

Date lipsă pentru un atribut într-un eșantion

Atribute numerice

Am observat faptul că atributul date_unregistration este extrem de important, astfel: dacă o linie conținea o valoare lipsă pe coloana date_unregistration, atunci automat studentul avea final_result Pass sau Distinction. Astfel, am imputat / transformat coloana astfel: dacă avea valoare lipsă am pus 0, iar altfel 1 (am pus 1 și nu am lăsat valoarea default, deoarece nu am vrut să fie afectată de testele de eliminare a outlierilor).

În rest, am folosit metoda cea mai simplă – am imputat valorile lipsă folosind media (univariată). Am folosit această metoda fiindcă este simplă și îmi pastrează media eșantionului.

Atribute categoricale

Am folosit cea mai simplă imputare (univariată) – valoarea cea mai frecventă. Această metodă merge pe premisa că datele care lipsesc, cel mai probabil, au valoarea cea mai frecventă. În plus, pastrează natura categoricală a datelor.

Valori extreme pentru un atribut într-un eșantion

z-score

Deși nu a fost menționat în enunțul temei, am ales să mă documentez despre testul z-score. Acesta ne spune cât de îndepărtat este un punct față de medie, în deviații standard. Dacă $z < -3$ sau $z > 3$, un punct este considerat outlier.

Acest test nu este util, deoarece distribuția nu este una normală și este sensibil la outlieri.

IQR (diferenta inter-quartila)

Aceasta metoda se uita la zona centrala a datelor (raspandirea zonei de mijloc - 50%, jumătatea din date care reprezintă comportamentul obișnuit).

Fata de z-score, aceasta metoda este rezistenta la outlieri si nu presupune o anume distributie.

Ca metoda de imputare, am ales sa folosesc mediana, deoarece nu este influentata de valori extreme, spre deosebire de medie.

Pentru acest set de date, eliminarea outlierilor nu este utila pentru sarcina de regresie, deoarece o parte din outlieri reprezinta informatie utila, nu zgomot.

Plaje valorice de mărimi diferite pentru attributele numerice

Anumiti algoritmi (spre exemplu regresia liniara / logistica) sunt afectati de disproportia scalelor. Astfel, este recomandat sa standardizam datele. Este important de mentionat, totusi, ca algoritmi finali utilizati: RandomForestClassifier si RandomForestRegressor nu sunt afectati de diferenta scalelor valorice. Am facut standardizarea atunci cand am folosit modele mai slabe.

Utilizarea algoritmilor de Învatare Automata

Clasificare

Pentru ca anumiti algoritmi produc erori atunci cand primesc input-uri categoriale, am ales sa convertesc input-urile intr-unele numerice folosind hot_encoder. Daca as fi folosit label encoding, spre exemplu, pentru coloana region, modelul ar putea transforma East Anglia in 1, Scotland in 2, Wales in 3 si asta ar afecta negativ regresia. One hot encoding creeaza o coloana separata pentru fiecare regiune.

Algoritm	max_depth	n_estimators	Modificari dataset	Acuratete	Precizie	Recall
DecisionTreeClassifier				0.669	0.671	0.669
DecisionTreeClassifier	5			0.737	0.754	0.737
RandomForestClassifier	15	300		0.757	0.760	0.757
RandomForestClassifier	15	300	plus imputare	0.755	0.755	<u>0.755</u>
RandomForestClassifier	15	300	plus outliers	<u>0.754</u>	<u>0.757</u>	0.754
RandomForestClassifier	15	300	plus standarizare	<u>0.754</u>	<u>0.757</u>	0.754
RandomForestClassifier	15	300	plus eliminarea atributelor redundante	<u>0.754</u>	<u>0.755</u>	0.754

Acest tabel reprezinta jurnalizarea experimentelor facute.

Prima linie reprezinta baseline-ul.

A doua iteratie a fost limitarea adancimii arborelui de decizie, acest lucru a fost foarte important deoarece a prevenit overfit-ul.

In urmatoarea iteratie, am folosit un model superior: Paduri aleatoare. Acest model combina mai multe modele de invatare pentru a creste acuratetea invatarii. Paduri aleatoare reprezinta un algoritm de tip bagging unde pentru fiecare dintre cele K modele:

- selectam aleator din SI (setul de invatare) o submultime de obiecte (exemple)
- selectam din A (setul de attribute) o submultime de attribute
- construim un arbore de decizie pe baza selectiilor realizate

Daca N este numarul total de exemple de invatare ($N = |SI|$) si M este numarul total de attribute ($M = |A|$) => Se vor selecta aleator pentru construirea fiecarui arbore m attribute ($m < M$) din A si n exemple ($n < N$) din SI. Restul exemplurilor se folosesc pentru evaluare. Fiecare arbore este construit complet plecand de la n si m.

Clasa finala se stabileste pe baza unui vot majoritar.

Avantaje paduri aleatoare vs ID3:

- acuratete mai mare la invatare (in seturi de date cu separabilitate complexa a claselor)
- mai robust la zgomotele din setul de invatare
- prezinta inca tendinte de overfitting dar mult mai atenuate decat ID3

Se poate observa, in tabel, ca a treia iteratie are rezultatele „cele mai bune” pe setul de validare. Trebuie, totusi, sa tinem cont ca setul de validare reprezinta 20% din setul original de train. Are rezultate bune pentru ca memoreaza zgomotul, insa penultima (iteratia curenta + imputare + outliers + standardizare) este cea mai relevanta. Acest lucru a fost putut vazut si la scorul de pe Kaggle.

In tabel, diferenta majora, dupa cum se poate observa, a fost facut de schimbarea algoritmului din DecisionTreeClassifier in RandomForestClassifier.

La DecisionTreeClassifier am pus max_depth mic (5) pentru a nu face overfitting, in schimb la RandomForestClassifier l-am lasat mai mare (15), deoarece setul de date este impartit intre acesti arbori, iar efectele overfitting-urilor individuale sunt mediate. Am folosit n_estimators = 300, pentru a avea un echilibru. Cu cat sunt mai multi estimatori, scade varianta si se reduce overfitting-ul. Nu am ales un numar mai mare, deoarece eroarea incepe sa stagneze, adica sa nu se modifice semnificativ, iar timpul pentru antrenare creste semnificativ.

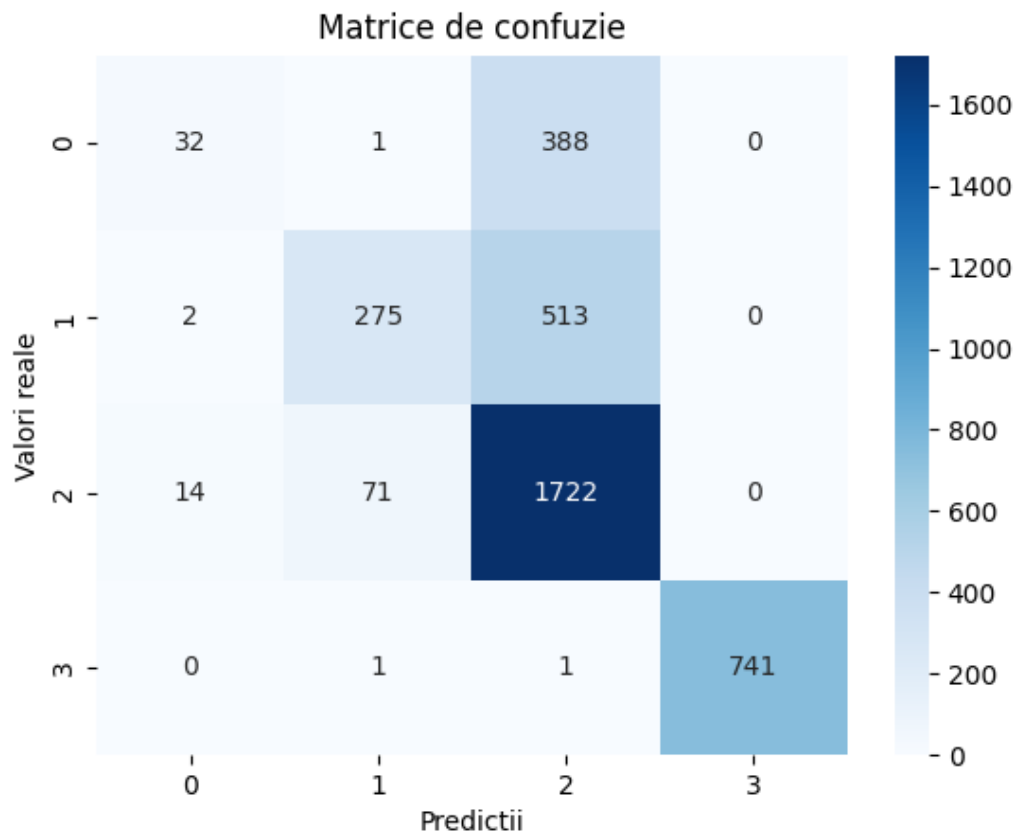
Statistici set de test (am putut face acest lucru, deoarece etichetele n-au fost ascunse, desi in enunt scrie clar ca sunt)

Accuracy: 0.7574468085106383

Precision: 0.7702605215573163

Recall: 0.7574468085106383

F1 Score: 0.7328737832751226



Matrice confuzie DecisionTreeClassifier (max_depth = 5)

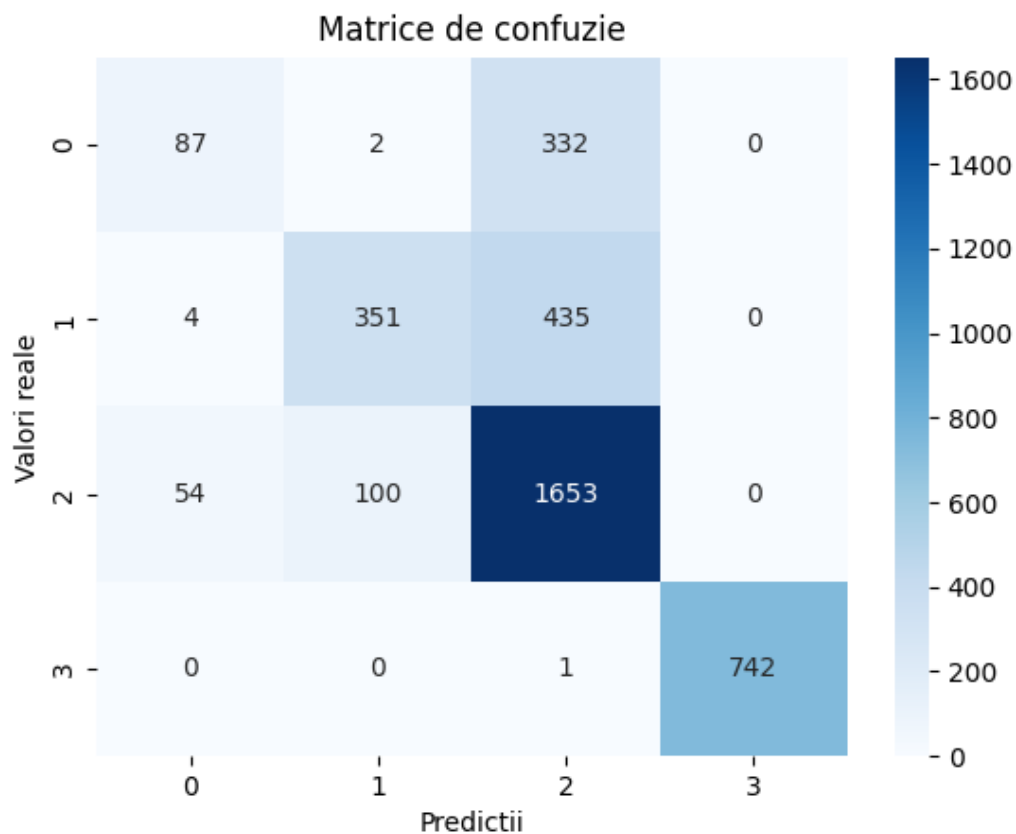
Cum am specificat si mai sus in document, am realizat transformarea: Withdrawn = 0, Fail = 1, Pass = 2, Distinction = 3.

Putem observa confuziile majore:

- in 513 cazuri algoritmul a prezis Pass cand era Fail (greseala semnificativa)
- in 388 din cazuri algoritmul a prezis Pass cand era Withdrawn (alta greseala importanta)

Exista un bias clar catre Pass. Acest lucru este datorat dezechilibrului datelor (sunt mai multi studenti care trec, decat care pica).

Putem observa si ca modelul a prezis distinction corect in majoritatea cazurilor, deci inseamna ca studentii cu distinction se comporta semnificativ diferit fata de restul.



Matrice confuzie RandomForestClassifier (max_depth = 15, n_estimators = 300)

Putem observa confuziile majore:

- in 435 cazuri algoritmul a prezis Pass cand era Fail (greseala semnificativa)
- in 332 din cazuri algoritmul a prezis Pass cand era Withdrawn (alta greseala importanta)

Se observa o diferenta semnificativa pozitiva fata de algoritmul precedent. Spre exemplu, la withdrawn a crescut numarul semnificativ. La fail a crescut, la fel, acuratetea.

Exista un bias clar catre Pass. Acest lucru este datorat dezechilibrului datelor (sunt mai multi studenti care trec, decat care pica).

Putem observa si ca modelul a prezis distinction corect in majoritatea cazurilor, deci inseamna ca studentii cu distinction se comporta semnificativ diferit fata de restul.

Clasa	Precizie (Precision)	Recall	F1-Score
Withdrawn (0)	0.60	0.21	0.31
Fail (1)	0.77	0.44	0.56
Pass (2)	0.68	0.91	0.78
Distinction (3)	1.00	1.00	1.00

Valorile de mai sus sunt cu aproximatie.

Regresie

Algorithm	Modificari dataset	alpha	n_estimators	max_depth	MAE	MSE	RMSE	R2
Linear Regression	imputare				<u>5.02</u>	<u>51.16</u>	7.15	0.82
Linear Regression	plus outliers (z-score si IQR)				6.93	114.98	10.72	<u>0.6</u>
Linear Regression	plus standardizare				<u>5.02</u>	51.14	7.15	0.82
Ridge	ca mai sus	1			<u>5.02</u>	<u>51.16</u>	7.15	0.82
RandomForestRegressor			300	15	5.01	51.41	<u>7.17</u>	0.82
RandomForestRegressor	eliminare redundanta		300	15	5.01	51.46	<u>7.17</u>	0.82

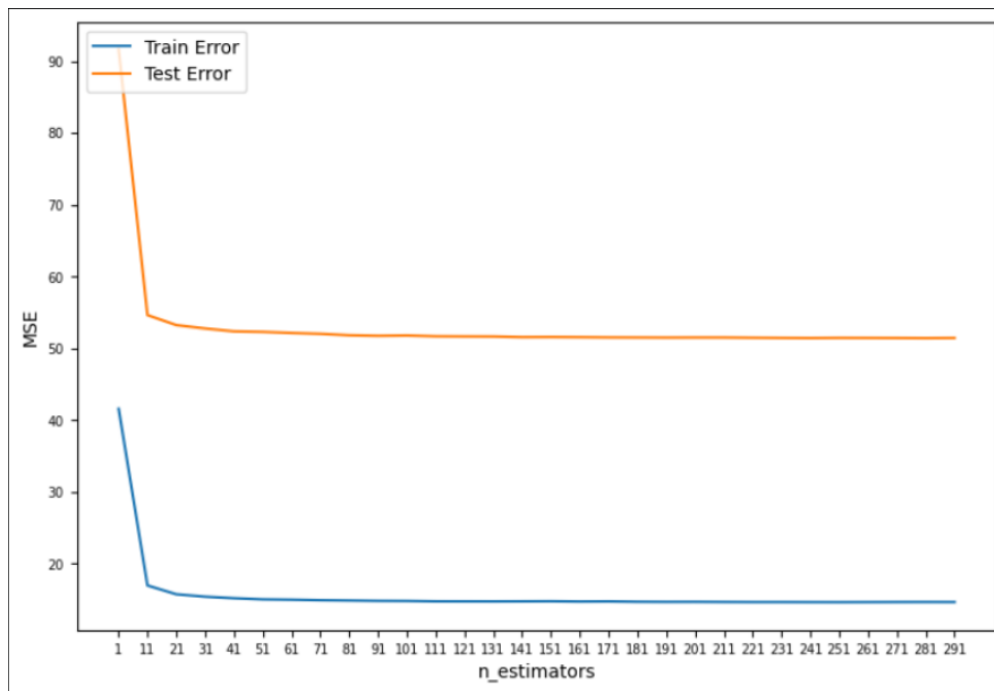
Acest tabel reprezinta jurnalizarea experimentelor facute.

Prima linie reprezinta baseline-ul. Desi observam ca eroarea este comparabila cu cel mai bun model, pe kaggle regresia liniara n-a fost alegerea buna. Acest lucru este datorat naturii diferite a algoritmului. Spre exemplu, regresia liniara este foarte sensibila la valori extreme, nefiind buna sa generalizeze.

A doua linie reprezinta tot modelul de regresie liniara, dar cu tot cu eliminarea outlierilor. Se observa o crestere drastica a erorii. Acest lucru este datorat faptului ca am eliminat niste evenimente rare, dar care nu reprezinta anomalie / zgomot.

Urmatorul model folosit este cel de Ridge cu alpha = 1, regresie cu regularizare, folosit pentru a generaliza, reducand overfitting-ul. Alpha = 1 inseamna o regularizare moderata, penalizand moderat coeficientii mari.

Cel mai bun model observat este, asemanator cu sarcina de clasificare RandomForestRegressor (n_estimators = 300 si max_depth = 15). Asa cum am mentionat si la clasificare, am ales max_depth = 15 si n_estimators = 300 pentru a reduce overfitting-ul. Nu am ales n_estimators mai mare, deoarece eroarea nu ar mai fi scazut semnificativ, insa timpul de antrenare ar fi crescut major.



Forma graficelor este asteptata. Se poate observa, atat pe setul de train, cat si pe cel de test, ca eroarea scade brusc, iar apoi se plafoneaza. Plafonarea si faptul ca graficul nu mai creste inseamna ca nu avem overfitting. Eroarea este mai mare pe setul de test, fata de cel de train, deoarece regulile extrem de specifice pe care le-a creat pe train nu vor generaliza de fiecare data.

Statistici set de test (am putut face acest lucru, deoarece etichetele n-au fost ascunse, desi in enunt scrie clar ca sunt)

MAE: 5.029371918627856

MSE: 51.54376305257142

RMSE: 7.179398516071623

R2: 0.806672881081247